

...Automatic Music Orchestration ...

Francesco Maccarini*

Affiliation
Address line 1
Address line 2
email

Mael Oudin

Affiliation
Address line 1
Address line 2
email

Gissel Velarde

Affiliation
Address line 1
Address line 2
email

Abstract

We present a symbolic automatic music orchestration method that learns an orchestration style from MIDI and transfers it to a target piano score. The baseline method formulates orchestration as note-wise supervised classification from note features (onset, duration, pitch, velocity) to an instrument label. We then extend this baseline in two directions. First, we introduce a doubling-aware preprocessing and postprocessing pipeline that compresses duplicated/transformed note events during training and reconstructs them at inference. Second, we introduce a multi-label extension in which each note can be assigned to multiple instruments using multi-hot targets and multi-output classification. The resulting framework supports both standard one-note/one-instrument orchestration and richer one-note/many-instruments textures.

FM: « Most of the text is now an ai-generated description of the repository. I need to rework most of it. »

1 INTRODUCTION

TODO: - What is orchestration

- Orchestration is more than instrumentation and instrument estimation (but you need to start somewhere)

Automatic orchestration maps a symbolic score to an instrumental realization. In this work, we target MIDI-to-MIDI orchestration transfer: given an orchestral reference file (style source) and a piano target file, we predict orchestral assignments for each target note while preserving the target's temporal structure.

The core challenge is that orchestration is not always one-to-one. A pitch event may be realized by several instruments (doublings), and equivalent events may appear through transposition-like transformations. A plain single-label classifier cannot represent these behaviors directly. We therefore start from a simple supervised baseline and progressively add two capabilities: doubling-aware processing and multi-label prediction.

2 STATE OF THE ART

TODO: - Focus on symbolic orchestration methods

- Frameworks: different tasks, how they go together, fully autonomous end-to-end vs co-creative. I think here we have a process in steps

- Distinguish from Orchidea and other sound imitation orchestration methods

- Present Gissel's Orchestration by Example (ObyE): orchestrate by learning from a model example, use tabular data representation, avoid data-hungry methods and use tree-based classification. Highlight advantages. Highlight problems addressed here.

ObyE provided a lightweight method to estimate the instrumentation of a piece. It does not require large training

datasets (only one example file) and it does not need long training times. This method however only allows to assign existing notes to instruments, and lacks the possibility of transforming the existing note content or creating new notes. This is a fundamental aspect of professional human orchestration, that heavily rely on "rewriting" rather than distributing the music content.

3 METHODS

3.1 Data Representation

Each MIDI note event is represented as an 8-column row:

[track number, track name, channel, program, onset, duration, pitch, velocity]

For prediction, features are:

$$\mathbf{x} = (\text{onset}, \text{duration}, \text{pitch}, \text{velocity}).$$

The label target is either:

- track-channel (e.g., "9_13"), or
- program (MIDI program number).

A mapping from predicted label back to full orchestral metadata is learned from training data (*quaterna mapping*), enabling reconstruction of track, track name, channel, program after prediction.

3.2 Baseline AMO (No Doublings, No Multi-Label) (ObyE)

3.2.1 Training

Given source orchestral MIDI:

1. Convert MIDI to note matrix and sort by onset, duration, and track.
2. Learn label-to-metadata mapping from the first four columns.
3. Build feature matrix X and label vector y .
4. Encode y with a label encoder.
5. Train a classifier $f : X \rightarrow y$ (e.g., XGBoost, Random Forest, Decision Tree, KNN, MLP, Naive Bayes, AdaBoost).

*Use footnote for providing further information about author (webpage, alternative address)—not for acknowledging funding agencies.

- Evaluate on held-out split; retrain on full data for final inference model.

3.2.2 Inference

Given target piano MIDI:

- Extract target features X^* .
- Predict labels $\hat{y} = f(X^*)$.
- Decode labels and reconstruct full metadata via mapping.
- Concatenate metadata with original note features and write orchestrated MIDI.

Timing and global musical structure are preserved from the target file in MIDI export.

3.3 Doubling-Aware Extension

3.3.1 Reduction During Preprocessing

To model instrumental doublings, notes are grouped by key

$$k = (\text{onset}, \text{pitch}),$$

with duration tolerance τ . If two notes match under this criterion, one is kept as seed and the other is dropped; a doubled counter is incremented on the seed note. Optionally, user-defined transformations (e.g., transposition by fixed semitones) are also tested; matched transformed notes increment transformation-specific counters.

This produces a reduced representation that captures:

- direct duplicate density (`doubled`),
- transformation-related density (one counter per transformation).

3.3.2 Transformation Classification

For each transformation t_j , a binary classifier is trained to predict whether that transformation should be applied to a target event. This is learned on reduced orchestral data and then applied to the target file.

3.3.3 Expansion During Inference

After transformation predictions:

- Duplicate seed notes according to `doubled`.
- Recreate transformed notes using inverse transformation calls.

This restores orchestral density before instrument assignment, allowing the model to reproduce style-specific doubling and transformation patterns.

3.4 Multi-Label (Multi-Hot) Extension

3.4.1 Why Multi-Label

Baseline classification enforces one label per note. Real orchestration often requires several instruments on one note event. We therefore replace single-label targets with multi-hot vectors.

3.4.2 Target Construction

On source data, notes are reduced by (onset, pitch). For each reduced seed note i , all associated instrument labels are aggregated:

$$Y_i \subseteq \mathcal{C}.$$

Using a `MultiLabelBinarizer`, each Y_i is converted to

$$\mathbf{y}_i \in \{0, 1\}^{|\mathcal{C}|}.$$

3.4.3 Model

A multi-output classifier is trained:

$$F : \mathbf{x}_i \mapsto \mathbf{y}_i.$$

Implementation uses `MultiOutputClassifier(base_clf)`, i.e., one binary predictor per class dimension.

3.4.4 Inference

For each target note feature vector:

- Predict multi-hot activations.
- Inverse-transform to active label set.
- Duplicate the note once per active label.
- Map each label back to full orchestral metadata.

This directly enables one-note/many-instruments orchestration at prediction time.

4 TRAINING AND INFERENCE SUMMARY

4.1 Training (Full System)

- Parse source MIDI (or list of source MIDI).
- Optional reduction with doubling/transformation counters.
- Build mapping for metadata reconstruction.
- Either:
 - single-label target (`multiclass=False`), or
 - multi-hot target (`multiclass=True`).
- Train instrument classifier.
- If transformations are enabled, train one classifier per transformation target.

4.2 Inference (Full System)

- Parse target MIDI.
- If transformations are enabled, predict transformation activations and expand target note list.
- Predict instrument assignments:
 - one label per note (single-label mode), or
 - multiple labels per note (multi-label mode, with note duplication).
- Reconstruct full MIDI metadata from predicted labels.
- Save orchestrated MIDI with preserved timing/meta structure.

TODO: Francesco: Continue this description

TODO: Add a scheme of the algorithm

TODO: Francesco: how can we make this better with additional features?

TODO: Francesco: how can we make this better now that we have a way to align the piano and orchestral pieces in the training dataset?

5 WHAT THE EXTENSIONS ENABLE

5.1 Baseline Capability

Baseline AMO learns a direct mapping from note features to one instrument label. It is simple and effective for monophonic assignment per event.

5.2 With Doublings

The model can represent and regenerate orchestral density patterns by learning where duplicate or transformed companion notes tend to occur.

5.3 With Multi-Label

The model can output several instruments for the same note event, enabling realistic doublings and thicker orchestral textures not possible in single-label mode.

6 RESULTS

FM: « We are not completely happy with what comes out of the model but we find something interesting »

TODO: Report the accuracy, precision, recall, f1 data and figure. Compare models

The approach is intentionally modular: baseline classification, doubling-aware reduction/expansion, and multi-label assignment can be enabled independently. This makes the framework usable as a plain orchestrator or as a richer texture-aware orchestrator depending on task requirements.

Potential limitations include note-wise independence, restricted feature set (no explicit harmonic context), and dependence on consistent label mapping in source files.

7 DISCUSSION

TODO: Discuss what is learned.

FM: « I believe that the model is learning the "structure" of the example piece rather than learning something really about orchestration. Different example pieces produce very different tree structures. However, this can be used if we are careful to provide example pieces that resemble the piece that we want to orchestrate. »

TODO: Report some figures with different pieces to show how notes are classified based on the features.

TODO: Francesco: What would these figures become if we train on the entire dataset?

TODO: Francesco: can the alignment be improved by considering that some repetitions might be expanded in one version?

8 CONCLUSION

We described a supervised MIDI orchestration framework that progresses from a simple single-label baseline to a doubling-aware, multi-label system. The baseline handles direct note-to-instrument transfer; doubling-aware processing adds texture reconstruction; and multi-hot multi-output classification enables multiple simultaneous instrument assignments per note. Together, these components provide a practical and extensible pipeline for automatic symbolic orchestration.

AUTHOR DECLARATIONS

Author declarations should include conflicts of interest, ethical issues, and comments on the use of AI.

Use unnumbered first level headings for the acknowledgments. All acknowledgments go at the end of the paper before the list of references.

Do **not** include acknowledgments in the anonymized submission, only in the final paper. You can use the `ack` environment provided in the style file to automatically hide this section in the anonymized submission.

REFERENCES

References follow the author declarations. Use unnumbered first-level heading for the references. Citation style should follow APA. It is permissible to reduce the font size to `small` (9 point) when listing the references.